

Ansible

Gestión de la configuración y automatización

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 PI · Puesta en Producción de Aplicaciones

PI · ANSIBLE

01

Introducción a Ansible

Qué es, instalación y primeros pasos

¿Qué es Ansible?

*" Ansible es una herramienta de **gestión de configuración y automatización** que permite definir, en ficheros de texto **YAML**, el estado deseado de servidores y aplicaciones, y aplicarlo **sin instalar agentes** en los nodos (usa **SSH**).*

CARACTERÍSTICAS

- Desarrollado principalmente por **Red Hat**
- Escrito en **Python** · disponible en PyPI
- Arquitectura **push** sin agentes — solo SSH
- Configuración declarativa en **YAML**
- Primera versión: **2012**

¿PARA QUÉ SIRVE?

- Configurar servidores de forma automática
- Instalar y gestionar paquetes y servicios
- Desplegar aplicaciones
- Orquestar tareas en **múltiples máquinas** a la vez
- Gestionar actualizaciones del sistema

Ansible en el contexto de IaC

La **Infraestructura como Código (IaC)** define y gestiona la infraestructura mediante ficheros de configuración en lugar de hacerlo manualmente. Existen dos tipos de herramientas:

SOFTWARE DE ORQUESTACIÓN

Programa la **creación de escenarios**: máquinas virtuales, redes, almacenamiento...

Ejemplo: OpenTofu

SOFTWARE DE GESTIÓN DE LA CONFIGURACIÓN (CMS)

Programa la **configuración que van a tener los servidores** una vez creados.

Ejemplo: Ansible

Instalación de Ansible

CON APT (DEBIAN/UBUNTU)

```
apt install ansible
```

Instala la versión empaquetada por la distribución.

CON PIP EN ENTORNO VIRTUAL (RECOMENDADO)

```
python3 -m venv ansible-env  
source ansible-env/bin/activate  
pip install ansible
```

Permite tener la **versión más reciente** e independiente del sistema.



Ansible solo necesita instalarse en el **nodo de control** (tu máquina). Los nodos gestionados únicamente requieren Python y acceso SSH.

El inventario

El **inventario** es el fichero donde definimos los equipos que Ansible va a gestionar. Los equipos se organizan en **grupos**.

```
all:
  children:
    servidores:
      hosts:
        nodo1:
          ansible_ssh_host: 192.168.1.10
          ansible_ssh_user: usuario
          ansible_ssh_private_key_file: ~/.ssh/id_rsa
```

- `all` — grupo raíz que engloba todos los equipos
- `servidores` — grupo personalizado con nuestros hosts
- Cada host puede tener variables propias: IP, usuario SSH, clave privada...

Fichero de configuración

Junto al inventario se crea un fichero `ansible.cfg` que configura el comportamiento de Ansible para el proyecto:

```
[defaults]
inventory = hosts
host_key_checking = False
```

- `inventory` — ruta al fichero de inventario
- `host_key_checking = False` — evita la verificación de la clave del host en cada conexión SSH (útil en entornos de pruebas)



Ansible busca este fichero en el directorio de trabajo. Cada proyecto puede tener su propia configuración.

Módulos y comandos ad-hoc

Un **módulo** permite ejecutar una acción concreta en uno o varios servidores remotos.

Los **comandos ad-hoc** ejecutan un módulo directamente desde la línea de comandos, sin necesidad de un playbook:

```
ansible <hosts> -m <módulo> -a "<parámetros>"
```

PARTE	DESCRIPCIÓN
<hosts>	<code>all</code> , nombre de grupo o nombre de host del inventario
-m <módulo>	módulo a usar
-a "<parámetros>"	argumentos del módulo
--become	ejecutar como <code>root</code> en el nodo remoto

Módulos esenciales (I)

PING

Comprueba la conectividad con los nodos. No es un ping ICMP, sino una verificación de conexión SSH y Python.

```
ansible all -m ping
ansible servidores -m ping
```

COMMAND / SHELL

Ejecuta comandos en el nodo remoto. `shell` permite pipes, redirecciones y variables de entorno.

COPY

Copia ficheros desde el nodo de control al nodo remoto.

- `src` — fichero origen (local)
- `dest` — ruta de destino (remoto)
- `mode` — permisos (opcional)

```
ansible all -m copy \
  -a "src=./index.html dest=/tmp/index.html mode=0644"
```

Módulos esenciales (II)

APT

Instala, actualiza o elimina paquetes en sistemas Debian/Ubuntu.

- `name` — paquete
- `state` — `present`, `absent`, `latest`

```
ansible nodo1 -m apt \  
  -a "name=apache2 state=present" --become
```

SERVICE

Gestiona servicios del sistema.

- `name` — servicio

FILE

Gestiona ficheros, directorios y permisos.

- `path` — ruta en el nodo remoto
- `state` — `file`, `directory`, `absent`, `link`
- `mode` — permisos

```
ansible all -m file \  
  -a "path=/tmp/demo state=directory mode=0755"
```

USER

Crea, modifica o elimina usuarios.

- `name` — nombre del usuario

Declarativo e idempotencia

ENFOQUE DECLARATIVO

Ansible **no** usa un esquema imperativo (*"instala apache"*).

Declaramos el **estado deseado** del servidor:

*"Quiero que el servidor tenga
apache2 instalado"*

Ansible hará todas las operaciones necesarias para que ese estado se cumpla.

IDEMPOTENCIA

Si el estado declarado **ya se ha alcanzado**, Ansible no ejecuta ninguna operación adicional.

- Primera ejecución: instala apache → salida en **amarillo** (`changed`)
- Segunda ejecución: ya está instalado → salida en **verde** (`ok`)

Podemos ejecutar el mismo playbook **múltiples veces** con total seguridad.

PI · ANSIBLE

¡Gracias!

Ansible · Introducción

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 PI